

# Native Graph Neural Network Training in MillenniumDB

Benito Fuentes<sup>1,3,4,\*</sup>, Sebastián Vergara<sup>1,3,4</sup> and Sebastián Ferrada<sup>2,3,4</sup>

<sup>1</sup>Department of Electrical Engineering, Data and Artificial Intelligence Initiative, Universidad de Chile, Santiago, Chile

<sup>2</sup>Department of Computer Science, Data and Artificial Intelligence Initiative, Universidad de Chile, Santiago, Chile

<sup>3</sup>Millennium Institute for Foundational Research on Data (IMFD), Chile

<sup>4</sup>National Center for Artificial Intelligence Research (CENIA), Chile

## Abstract

Training Graph Neural Networks on graphs that exceed main memory often requires exporting data from graph database systems to external machine learning frameworks. We present a database-native, out-of-core GraphSAGE training pipeline integrated into MillenniumDB and exposed through four GQL procedures. The pipeline materializes reusable mini-batches, distributes features across GPU memory, pinned host memory, and NVMe storage based on access frequency, applies frequency-based tiering to the graph topology as well, and writes the learned embeddings back into MillenniumDB for queries that combine graph patterns with similarity search. We evaluate on Cora, ogbn-arxiv, ogbn-products, and ogbn-papers100M (111M nodes, 1.6B directed edges). On a commodity desktop (32 GiB RAM, 16 GB GPU), PyG, DGL, and Neo4j GDS run out of memory on ogbn-papers100M. MillenniumDB completes 50 training epochs in 41.2 minutes after a one-time 95.7-minute preparation, matching the accuracy of the state-of-the-art out-of-core system DiskGNN. An ablation shows that topology tiering speeds up offline sampling by  $3.81\times$  without altering the sampled mini-batches.

## 1. Introduction

Graph Neural Networks (GNNs) [1] learn representations from graph-structured data by aggregating information from local neighborhoods. They are widely used in applications such as molecular property prediction [2] and recommender systems [3]. Among GNN architectures, GraphSAGE [4] is particularly well-suited to large, evolving graphs because it learns an inductive aggregation function from sampled neighborhoods. Nevertheless, training GNNs at scale remains challenging: storing node features, intermediate embeddings, and the graph topology in GPU or host memory quickly becomes infeasible as graphs grow to millions of nodes and billions of edges [5].

DiskGNN [6] addresses this limitation with out-of-core training: it stores node features on disk, materializes neighborhood samples before training, and organizes data to reduce random I/O operations. DiskGNN and similar systems, however, are standalone machine learning pipelines rather than components of a graph database management system. Meanwhile, graph database systems (e.g., Neo4j with GDS [7]) typically either run graph algorithms and embedding methods in memory or require exporting the graph to external machine learning frameworks. As a result, scalable GNN training remains decoupled from the system that stores, indexes, and queries the graph.

We address this gap with a database-native, DiskGNN-inspired GraphSAGE training pipeline integrated into MillenniumDB [8, 9], a multimodel, multimodal graph DBMS with worst-case-optimal joins. The pipeline has four GQL procedures: graph projection, neighborhood sampling, feature store construction, and model training. All four access graph topology and node features through MillenniumDB’s storage and indexes. Mini-batches are sampled once and reused across training runs. Node features and graph topology are each kept in a four-tier store that places data by access frequency: the feature store serves training, and the topology store serves sampling.

Our contributions are threefold. First, we expose the complete GNN workflow as native GQL procedures that run over persistent graph projections without exporting the graph. Second, whereas

---

Alberto Mendelzon International Workshop on Foundations of Data Management, November 11–13, 2026, Arequipa, Perú

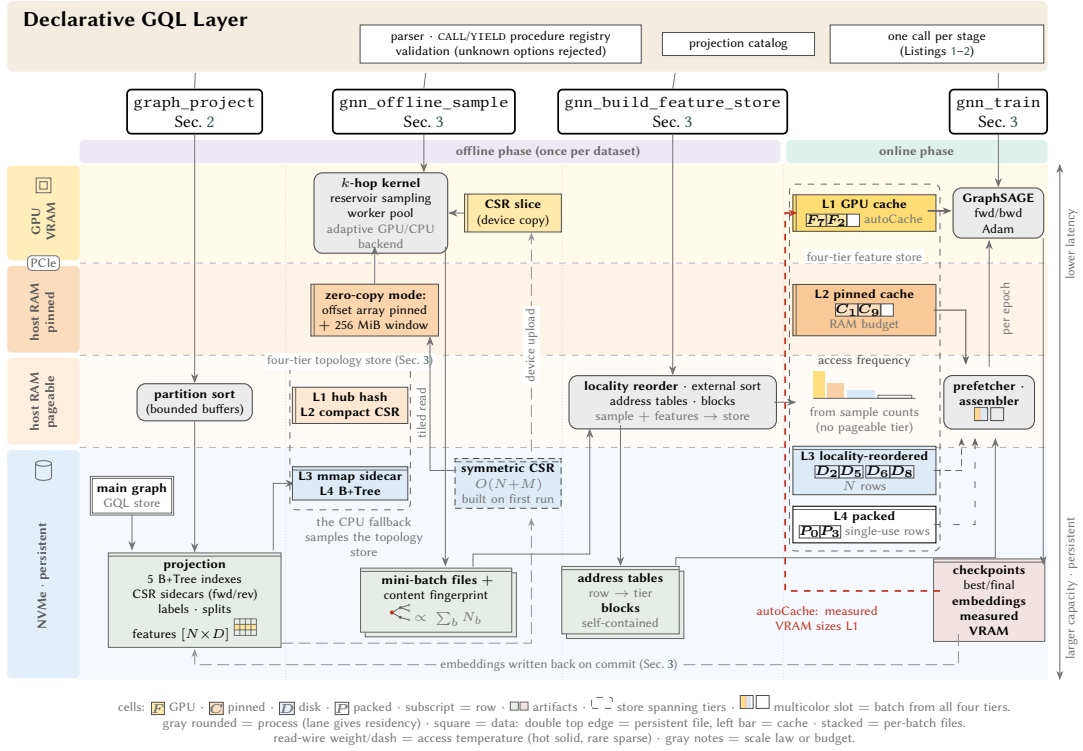
\*Corresponding author.

✉ benito.fuentes@ug.uchile.cl (B. Fuentes)

🆔 0009-0003-4668-580X (B. Fuentes); 0009-0005-3511-060X (S. Vergara); 0000-0002-9421-8566 (S. Ferrada)



© 2026 Copyright for this paper by its authors. Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).



**Figure 1:** Full view of the native GNN training pipeline: columns are the four GQL procedures in execution order, and horizontal lanes are the memory hierarchy (GPU VRAM to NVMe).

DiskGNN tiers only node features, we also tier the graph topology by access frequency. Third, we write the learned embeddings back into MillenniumDB, where they can be indexed for hybrid queries that combine graph patterns with similarity search [10].

We evaluate the pipeline on four supervised node classification datasets against PyG [11], DGL [12], DiskGNN, and Neo4j GDS. On ogbn-papers100M (over 111 million nodes), PyG, DGL, and Neo4j GDS run out of memory on a single commodity desktop. On the same machine, our pipeline matches the validation accuracy that DiskGNN reports on a 748 GB server, and DiskGNN’s reported test accuracy falls within the range of our ten runs. An ablation further shows that the four-tier topology store speeds up offline sampling by  $3.81\times$  without altering the sampled mini-batches.

## 2. Graph Projections

MillenniumDB stores and manages a persistent *main graph*. Before training, we extract from it the graph topology and features that GraphSAGE requires and materialize them as a *projection*: a named graph, computed once and stored on disk, that can be queried like any other graph in the system.

The `graph_project` procedure builds projections, following Neo4j GDS [7]. It takes a node label, an edge label, and a set of parameters, and keeps only the matching nodes and edges. The parameters attach input features and, as node-level columns, class labels and train/validation/test splits. They also select which indexes to build. A full projection creates ten B+Tree indexes to support arbitrary GQL queries, but GNN training requires only five: the node index and the two label maps, used to locate seed nodes, and the two adjacency-direction indexes, used for neighborhood sampling. Setting `indexSet` to `GNN_MINIMAL` builds only these five, which reduces build time and disk usage. Listing 1 shows this configuration for ogbn-papers100M [13], our running example.

Unlike the main graph, which stays durable and open to concurrent updates, a projection is static: it persists under its own name until deleted and is reused across training runs without recomputation. Several projections, each built and indexed for a specific task, can coexist.

```

1 CALL graph_project('papers100M', 'Node', 'CITES', {
2     indexSet: 'GNN_MINIMAL', includeFeatures: 'node_features',
3     labelProperty: 'label', splitProperty: 'split'
4 }) YIELD nodeCount, relationshipCount, featureDim, numClasses

```

Listing 1: Projecting the ogbn-papers100M citation graph with supervised metadata.

### 3. Disk-based GNN Training in MillenniumDB

The rest of the pipeline consists of three procedures, shown in Listing 2 and traced across the memory hierarchy in Figure 1. Sampling and feature store construction follow the out-of-core design of DiskGNN [6]. **Offline sampling.** `gnn_offline_sample` materializes every mini-batch once and stores it as a reusable file. For each seed batch, it expands  $k$ -hop neighborhoods with the configured fanouts on an adaptive GPU/CPU backend, respects the predefined train, validation, and test splits, and records a fingerprint of the sampled content so that runs can be compared. **Topology store.** The sampler reads adjacency lists from the four-tier topology store: an L1 hub hash and an L2 compact CSR in host memory hold the most frequently accessed lists, backed by an L3 mmap-backed CSR sidecar and the underlying B+Tree indexes (L4) on NVMe. **Feature store.** `gnn_build_feature_store` places node features by their access frequency in the materialized mini-batches: the most frequent rows go to an L1 GPU cache, which can be sized automatically from measured VRAM, the next to L2 pinned host memory within the RAM budget, the rest to an L3 locality-reordered NVMe layout, and rows accessed only once to an L4 packed layout. The store is persisted and reused across training runs. **Training and write-back.** `gnn_train` runs mini-batch GraphSAGE over the materialized samples and the feature store, with an asynchronous prefetcher that overlaps batch assembly and feature retrieval with the forward and backward passes. It keeps model checkpoints, and `writeProperty` writes the best or final embeddings back into MillenniumDB, where they can be indexed and combined with graph patterns in similarity queries [10].

```

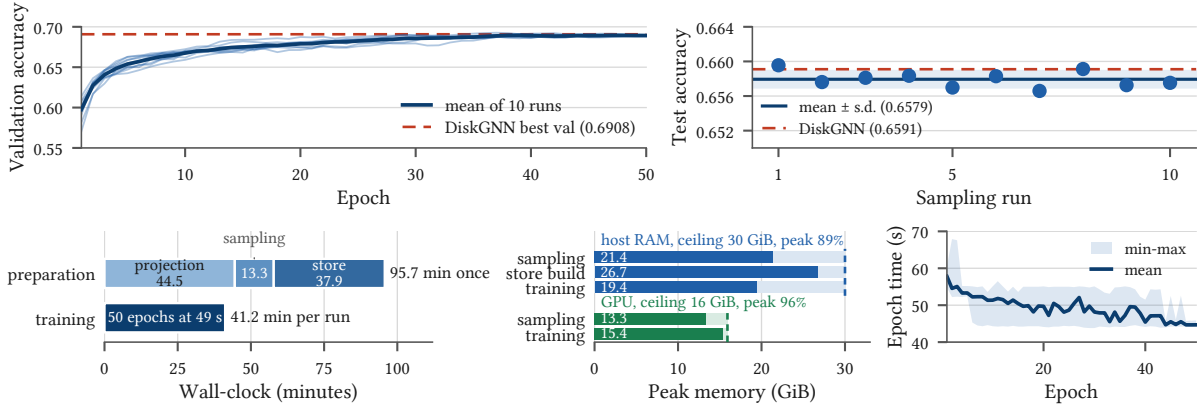
1 CALL gnn_offline_sample('papers100M', 'cites_sample', [10,15,20], {batchSize: 1024,
2     orientation: 'UNDIRECTED', usePredefinedSplits: true, randomSeed: 42, numWorkers: 16
3 }) YIELD totalBatches, uniqueNodes, sampleContentFp
4 CALL gnn_build_feature_store('cites_sample', 'node_features', {bakeBlocks: true,
5     autoCache: true
6 }) YIELD gpuCacheMb, cpuCacheMb, totalDiskMb
7 CALL gnn_train('cites_sample', 'node_features', {model: 'graphsage', hiddenDim: 256,
8     epochs: 50, lr: 0.001, dropout: 0.2, randomSeed: 42, writeProperty: 'embedding'
9 }) YIELD testAccuracy, bestValAccuracy, trainSeconds, l1HitRatio

```

Listing 2: The disk-based training pipeline on ogbn-papers100M.

### 4. Experimental Results

Every experiment is supervised node classification. We use Cora [14] (2.7K nodes, 5.3K edges) and three OGB graphs [13]: ogbn-arxiv (169K nodes, 1.2M edges), ogbn-products (2.4M nodes, 62M edges), and ogbn-papers100M (111M nodes, 1.6B directed edges, 53 GiB of features). The task is to predict a product’s category on ogbn-products and a paper’s research topic on the other three. All experiments, including the baselines PyG 2.8, DGL 2.5, Neo4j GDS 2026.06, and DiskGNN, ran on one commodity desktop (20-core Intel Core Ultra 7 265, 32 GiB RAM with about 30 GiB available, 16 GB NVIDIA RTX 5070 Ti GPU, NVMe drive). On ogbn-papers100M, MillenniumDB uses the configuration in Listing 2. **Scale.** On ogbn-papers100M, only MillenniumDB and DiskGNN complete training (Table 1). PyG exhausts memory before completing its first epoch (in two attempts) and DGL 61 s into dataset loading; in both cases, the Linux kernel kills the process. Neo4j GDS imports the graph in 14 min 55 s but exhausts a 24 GiB Java heap while projecting its 3.2 billion undirected relationships.



**Figure 2:** Breakdown of the ogbn-papers100M experiments. Top left: validation accuracy of a separate experiment with fixed sampled batches and ten training seeds. Top right: test accuracy of the ten sampling-seed runs with a fixed training seed. Dashed lines mark DiskGNN’s reported values. Bottom: wall-clock time, peak memory, and epoch time as the page cache warms up.

**Comparison to DiskGNN.** Across ten sampling seeds, MillenniumDB reaches a test accuracy of  $0.6579 \pm 0.0009$  at the best-validation epoch (range 0.6566–0.6596) and a best validation accuracy of  $0.6907 \pm 0.0011$ . DiskGNN, evaluated on a 748 GB server, reports 0.6591 test accuracy [6], which falls within our range; its released logs record 0.6908 best validation accuracy [15]. On our desktop, a single DiskGNN run reaches only 0.6504. We score it every fifth epoch with our own script, because DiskGNN’s shipped evaluator pins the whole 53 GiB feature matrix and is killed on both the GPU and the host; DiskGNN also fixes no seed, so its spreads in Table 1 reflect run-to-run variation. Figure 2 breaks down our cost: a one-time preparation of 95.7 min (projection, sampling, and feature store construction), amortized like an index build, then 41.2 min of training per run (49 s per epoch with a 9.2 GB feature cache). DiskGNN averages 62 s per epoch with its default 5 GB cache; measured cold over three epochs, its epoch time falls from 81 s at 5 GB to 63 s at 10 GB and 41 s at 15 GB, faster than our 49 s mean. Our peak usage reaches 96% of the GPU memory and 26.7 GiB (89%) of the available host memory, so tiering, not spare headroom, is what fits the features on this hardware.

**Comparison to general frameworks.** On the three smaller graphs, MillenniumDB, PyG, and DGL share the official splits, sampled inference, and a two-layer GraphSAGE-mean of width 256 trained with Adam under the same cosine learning-rate schedule. MillenniumDB trails the best framework by 0.0087 in accuracy on Cora, 0.0122 on ogbn-arxiv, and 0.0057 on ogbn-products (Table 1). Its end-to-end time (Table 2) includes import and index construction, whereas PyG and DGL read a preprocessed cache and DiskGNN converts such a cache into its own format. Even so, MillenniumDB is the fastest system on ogbn-arxiv:  $3.5\times$  faster than the faster framework and  $1.4\times$  faster than DiskGNN with a matched feature cache. On ogbn-products, it is at least  $2.4\times$  faster than both frameworks but  $1.3\times$  slower than DiskGNN, which packs only the training split, an eighth of the nodes. On Cora, fixed costs dominate runtimes of a few seconds. Neo4j GDS, whose times also include import, serves as a graph-DBMS reference; its embedding-plus-classifier pipeline is not equivalent to supervised GraphSAGE.

**The ogbn-arxiv gap.** Most of this gap comes from the output layer, not from database-native execution: MillenniumDB keeps the final convolution at hidden width and appends a linear classifier, whereas PyG and DGL place the class dimension directly on that convolution. Matching their placement in an 81-run ablation recovers 0.0106 of the 0.0122 difference ( $0.6961 \pm 0.0017$ ), improves Cora by 0.0254, and leaves ogbn-products unchanged. We report our original architecture, since the ablation is meant to explain the gap, not to retune the model.

**Topology-tiering ablation.** To isolate the contribution of topology tiering, we run offline sampling on ogbn-papers100M three times with it and three times with the legacy CSR topology path it replaces. Both use the same binary and the configuration of Listing 2, but fix the CPU backend and a per-node sampling strategy to isolate topology-access costs, which is why these times differ from the sampling

**Table 1**

Test accuracy at the best-validation epoch (mean  $\pm$  SD over ten runs; three for ogbn-products). Bold marks the best of our runs; <sup>†</sup> values reported in [6].

| System               | Cora                                | ogbn-arxiv                          | ogbn-products                       | ogbn-papers100M                     |
|----------------------|-------------------------------------|-------------------------------------|-------------------------------------|-------------------------------------|
| MillenniumDB         | 0.7901 $\pm$ 0.0121                 | 0.6855 $\pm$ 0.0052                 | 0.7633 $\pm$ 0.0008                 | <b>0.6579<math>\pm</math>0.0009</b> |
| PyG                  | <b>0.7988<math>\pm</math>0.0069</b> | <b>0.6977<math>\pm</math>0.0017</b> | 0.7647 $\pm$ 0.0017                 | OOM                                 |
| DGL                  | 0.7855 $\pm$ 0.0120                 | 0.6943 $\pm$ 0.0011                 | <b>0.7690<math>\pm</math>0.0009</b> | OOM                                 |
| DiskGNN              | 0.7969 $\pm$ 0.0036                 | 0.6953 $\pm$ 0.0016                 | 0.7551 $\pm$ 0.0015                 | 0.6504                              |
| DiskGNN <sup>†</sup> | –                                   | –                                   | –                                   | 0.6591                              |
| Neo4j GDS            | 0.6100 $\pm$ 0.0089                 | 0.6596 $\pm$ 0.0033                 | 0.6836 $\pm$ 0.0005                 | OOM in projection                   |

**Table 2**

Execution time: end-to-end on the three smaller graphs; one-time preparation, 50-epoch training, and their sum (first run) on ogbn-papers100M. n.m.: not measured separately.

| System       | End-to-end time |       |          | ogbn-papers100M   |          |           |
|--------------|-----------------|-------|----------|-------------------|----------|-----------|
|              | Cora            | arxiv | products | Preparation       | Training | First run |
| MillenniumDB | 4.7s            | 13.3s | 3.6m     | 95.7m             | 41.2m    | 136.9m    |
| PyG          | 3.7s            | 46.9s | 9.3m     | –                 | OOM      | –         |
| DGL          | 28.6s           | 50.9s | 8.7m     | OOM               | –        | –         |
| DiskGNN      | 12.1s           | 18.4s | 2.8m     | n.m.              | 51.5m    | n.m.      |
| Neo4j GDS    | 2.1m            | 21.2m | 97.1m    | OOM in projection | –        | –         |

time in Figure 2. Topology tiering reduces sampling time from  $62.03 \pm 20.83$  to  $16.30 \pm 0.53$  min, a  $3.81 \times$  speedup, and all six runs produce the same sample-content fingerprint. Profiling is consistent with this: the legacy path is dominated by small topology reads from NVMe, and in one matched pair of runs, tiering cuts physical reads from about 3.40 TB to 376.5 GB.

## 5. Conclusions

We have shown that out-of-core GNN training can be a database operation rather than an export. MillenniumDB trains GraphSAGE on ogbn-papers100M on a commodity desktop where PyG, DGL, and Neo4j GDS run out of memory, and matches DiskGNN’s reported validation accuracy; DiskGNN’s reported test accuracy falls within the range of our ten runs. On smaller graphs, it remains competitive in accuracy and is the fastest on ogbn-arxiv. Topology tiering speeds up offline sampling by  $3.81 \times$  without altering the sampled mini-batches. The natural next step is link prediction over projections, which requires leakage-safe edge splits, negative sampling, and a ranking-based decoder.

## Acknowledgments

This work was funded by ANID - Millennium Science Initiative Program - Code ICN17\_002; by CENIA FB210017, Basal ANID; by FONDECYT Grant 11251322; and by the Vicerrectoría de Investigación y Desarrollo (VID) of Universidad de Chile, project code: UI-016/24.

## Declaration on Generative AI

The authors used OpenAI ChatGPT with GPT-5.6 Thinking to assist with writing and editing, and Anthropic Claude Code with Claude Opus 4.8 to assist with code development and with Claude Opus 5.5 to assist with writing and editing. All generated outputs were reviewed and validated by the authors, who take full responsibility for the final content.

## References

- [1] F. Scarselli, M. Gori, A. C. Tsoi, M. Hagenbuchner, G. Monfardini, The Graph Neural Network Model, *IEEE Transactions on Neural Networks* 20 (2009) 61–80. URL: <http://ieeexplore.ieee.org/document/4700287/>. doi:10.1109/TNN.2008.2005605.
- [2] J. G. Rittig, Q. Gao, M. Dahmen, A. Mitsos, A. M. Schweidtmann, Graph Neural Networks for the Prediction of Molecular Structure–Property Relationships, in: *Machine Learning and Hybrid Modelling for Reaction Engineering*, Royal Society of Chemistry, 2023, pp. 159–181. doi:10.1039/BK9781837670178-00159.
- [3] S. Wu, F. Sun, W. Zhang, X. Xie, B. Cui, Graph neural networks in recommender systems: A survey, *ACM Comput. Surv.* 55 (2022). URL: <https://doi.org/10.1145/3535101>. doi:10.1145/3535101.
- [4] W. Hamilton, Z. Ying, J. Leskovec, Inductive Representation Learning on Large Graphs, in: *Advances in Neural Information Processing Systems*, volume 30, Curran Associates, Inc., 2017. URL: [https://proceedings.neurips.cc/paper\\_files/paper/2017/file/5dd9db5e033da9c6fb5ba83c7a7e9-Paper.pdf](https://proceedings.neurips.cc/paper_files/paper/2017/file/5dd9db5e033da9c6fb5ba83c7a7e9-Paper.pdf).
- [5] Z. Liu, K. Zhou, F. Yang, L. Li, R. Chen, X. Hu, EXACT: scalable graph neural networks training via extreme activation compression, in: *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25–29, 2022, OpenReview.net, 2022*. URL: [https://openreview.net/forum?id=vkaMaq95\\_rX](https://openreview.net/forum?id=vkaMaq95_rX).
- [6] R. Liu, Y. Wang, X. Yan, H. Jiang, Z. Cai, M. Wang, B. Tang, J. Li, DiskGNN: Bridging I/O Efficiency and Model Accuracy for Out-of-Core GNN Training, *Proc. ACM Manag. Data* 3 (2025). URL: <https://doi.org/10.1145/3709738>. doi:10.1145/3709738.
- [7] Neo4j, Inc., Neo4j Graph Data Science library, <https://neo4j.com/docs/graph-data-science/current/>, 2026. Accessed on July 2026.
- [8] D. Vrgoč, C. Rojas, R. Angles, M. Arenas, D. Arroyuelo, C. Buil-Aranda, A. Hogan, G. Navarro, C. Riveros, J. Romero, MillenniumDB: An Open-Source Graph Database System, *Data Intelligence* 5 (2023) 560–610. URL: <https://direct.mit.edu/dint/article/5/3/560/117375/MillenniumDB-An-Open-Source-Graph-Database-System>. doi:10.1162/dint\_a\_00229.
- [9] D. Vrgoč, C. Rojas, R. Angles, M. Arenas, V. Calisto, B. Farías, S. Ferrada, T. Heuer, A. Hogan, G. Navarro, A. Pinto, J. Reutter, H. Rosales, E. Toussaint, MillenniumDB: A Multi-modal, Multi-model Graph Database, in: *Companion of the 2024 International Conference on Management of Data, ACM, Santiago, Chile, 2024*, pp. 496–499. URL: <https://dl.acm.org/doi/10.1145/3626246.3654757>. doi:10.1145/3626246.3654757.
- [10] V. Calisto, S. Ferrada, G. Navarro, J. L. Reutter, J. P. Sánchez, D. Vrgoč, Graph Querying or Similarity Search? Both!, in: *The Semantic Web – ISWC 2025*, volume 16140, Springer Nature Switzerland, Cham, 2026, pp. 349–368. doi:10.1007/978-3-032-09527-5\_19.
- [11] M. Fey, J. E. Lenssen, Fast graph representation learning with PyTorch Geometric, *arXiv preprint arXiv:1903.02428* (2019). URL: <https://arxiv.org/abs/1903.02428>.
- [12] M. Wang, D. Zheng, Z. Ye, Q. Gan, M. Li, X. Song, J. Zhou, C. Ma, L. Yu, Y. Gai, T. Xiao, T. He, G. Karypis, J. Li, Z. Zhang, Deep Graph Library: A graph-centric, highly-performant package for graph neural networks, *arXiv preprint arXiv:1909.01315* (2019). URL: <https://arxiv.org/abs/1909.01315>.
- [13] W. Hu, M. Fey, M. Zitnik, Y. Dong, H. Ren, B. Liu, M. Catasta, J. Leskovec, Open Graph Benchmark: Datasets for machine learning on graphs, in: *Advances in Neural Information Processing Systems* 33 (NeurIPS 2020), 2020.
- [14] P. Sen, G. Namata, M. Bilgic, L. Getoor, B. Gallagher, T. Eliassi-Rad, Collective classification in network data, *AI Magazine* 29 (2008) 93–106. doi:10.1609/aimag.v29i3.2157.
- [15] R. Liu, et al., DiskGNN source code and experiment logs, <https://github.com/Liu-rj/DiskGNN>, 2026. Accessed on September 2026.